`std::is_debugger_present`

René Ferdinand Rivera Morell – grafikrobot@gmail.com – P2515R0, 2021-12-29

Table of Contents

Document number:

ISO/IEC/JTC1/SC22/WG21/P2515R0

Date:

2021-12-29

Audience:

SG15, LEWG

Reply-to:

René Ferdinand Rivera Morell, grafikrobot@gmail.com

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

1. Abstract

This paper proposes a new function, `std::is_debugger_present`, that checks if a program is being debugged to aid in software development.

2. Revision History

2.1. Revision 0 (January 2022)

Initial text.

3. Motivation

There are many scenarios where doing something special when your program is running in a debugger is important. That can take the form of:

- allowing printing out extra output to help diagnose problems,

- executing extra test code,
- displaying an extra user interface to help in debugging,
- and more.

This is something that appears in many development environments but is hard to do as it requires intimate platform knowledge. Making this facility available would improve the program debugging experience of the average user. And spare them the burden of acquiring arcane platform knowledge to implement, over and over, this functionality.

4. Design Decisions

The goal of the `std::is_debugger_present` function is to inform when a program is executing under the control of a debugger monitoring program. The interface is minimally simple to avoid having to reduce the user from having to know the intricacies of debugger operation. This is a feature that requires arcane platform knowledge for most platforms. But it is knowledge that is readily available to the platform tooling implementors.

The `std::is_debugger_present` function is intended to go into a `<debugging>` header.

4.1. Impact On the Standard

This proposal adds a utility header and a single function the implementation of which is readily available to the platform tooling implementors.

5. Implementation Experience

In addition to the prototype implementation [1] there are the following, full or partial, equivalent implementations:

- Windows provides a `IsDebuggerPresent` function in the OS with the same functionality. [2]
- Unreal Engine 4 implements the same as a `IsDebuggerPresent` function for a variety of platforms.
- Catch2 implements the same as a `isDebuggerActive` function. [3]

6. Wording

Wording is relative to N4868 (https://wg21.link/N4868). [4]

6.1. Library

Add a new entry to General utilities library summary [utilities.summary] table.

[debugging]	Debugging	`<debugging>`

Add section to General utilities library [utilities].

6.1.1. Debugging [debugging]

6.1.1.1. In general [debugging.general]

Subclause [debugging] describes the debugging library that provides functionality to introspect and interact with a debugger that is executing and monitoring the running program.

6.1.1.2. Header `<debugging>` synopsis [debugging.syn]

```cpp
namespace std {
    // [debugging.utility], utility
    bool is_debugger_present() noexcept;
}
```

6.1.1.3. Utility [debugging.utility]

`bool is_debugger_present() noexcept;`

Returns: If the program is currently running under a debugger it returns `true`, otherwise `false`.

Remarks: If not supported by the implementation it returns `false`.

7. Acknowledgements

1. Debugging prototype implementation (https://github.com/grafikrobot/debugging)
2. Win32 IsDebuggerPresent (https://docs.microsoft.com/en-us/windows/win32/api/debugapi/nf-debugapi-isdebuggerpresent)
3. Catch2 isDebuggerActive (https://github.com/catchorg/Catch2/blob/devel/src/catch2/internal/catch_debugger.cpp)
4. N4868 Working Draft, Standard for Programming Language C++ 2020-10-18 (https://wg21.link/N4868)